

Excellent choice. A **Resume Search Engine** is the next logical project after your FAQ Search Engine because it reuses many of the same NLP concepts while introducing document ranking and search.

Since you're an experienced C# developer, we'll build it without external AI libraries first so you understand the core algorithms.

Project 2: Resume Search Engine

Goal

Suppose HR has 10,000 resumes.

Recruiter searches:

"Senior C# Developer with SQL Server and Azure"

Your system should rank resumes like:

```
1. Resume_15.pdf    Score: 0.91
2. Resume_08.pdf    Score: 0.83
3. Resume_41.pdf    Score: 0.79
...
```

Exactly how job portals perform resume searching (at a basic level).

What you will learn

This project teaches:

- Document Processing
- Information Retrieval
- Search Ranking
- TF-IDF
- Cosine Similarity
- Multi-document search
- Score calculation
- Keyword extraction
- Resume indexing

These are the foundations for semantic search and RAG.

Project Architecture

Resume Folder



Read Resume Files



Text Preprocessing



Tokenization



Stopword Removal



Lemmatization



Build Vocabulary



TF-IDF Vector for Every Resume



Save Resume Vectors



User Search Query



Preprocess Query



TF-IDF Vector



Cosine Similarity



Sort Descending



Folder Structure

ResumeSearchEngine

Models

ResumeModel.cs

Services

TextProcessingService.cs

TFService.cs

IDFService.cs

TFIDFService.cs

VectorService.cs

SearchService.cs

Data

resume1.txt

resume2.txt

resume3.txt

resume4.txt

Later you can add PDF support.

ResumeModel

```
public class ResumeModel
{
    public int Id { get; set; }

    public string FileName { get; set; }

    public string Content { get; set; }

    public Dictionary<string,double> TFIDF { get; set; }
```

```
public double Score { get; set; }  
}
```

Sample Resume

Name : John

Experience

Senior C# Developer

Skills

C#

ASP.NET Core

SQL Server

Azure

Docker

REST API

Another

Name : David

Experience

Java Developer

Skills

Java

Spring Boot

MySQL

Hibernate

AWS

Search Example

User types

Senior C# Developer SQL Server Azure

After preprocessing

```
senior
c#
developer
sql
server
azure
```

TF-IDF

Create TF-IDF vectors for every resume exactly as you did in the FAQ project.

```
Resume1
```

```
{
  c# : 0.41
  sql : 0.29
  azure : 0.52
}
```

```
Resume2
```

```
{
  java : 0.51
  spring : 0.40
  mysql : 0.38
}
```

Query Vector

Build a TF-IDF vector for the user's search.

```
Query
```

```
c#
sql
azure
```

Cosine Similarity

Calculate similarity between the query and every resume.

```
Resume1 = 0.93
```

```
Resume2 = 0.18
```

```
Resume3 = 0.72
```

```
Resume4 = 0.44
```

Sort Results

```
Resume1
```

```
0.93
```

```
Resume3
```

```
0.72
```

```
Resume4
```

```
0.44
```

```
Resume2
```

```
0.18
```

Features to Build (Recommended Order)

Phase 1 (Core Search)

- Read resume text files
- Normalize text
- Tokenize
- Remove stop words
- Lemmatize
- Build vocabulary
- Calculate TF
- Calculate IDF
- Calculate TF-IDF
- Create vectors
- Cosine similarity

- Rank resumes
-

Phase 2 (Better Search)

- Skill extraction
 - Experience extraction
 - Education extraction
 - Email extraction
 - Phone extraction
-

Phase 3 (Advanced Filters)

Search like:

```
Skill = C#
```

```
Experience > 5 years
```

```
Degree = BSc
```

Phase 4 (Performance)

- Precompute TF-IDF once
 - Cache vectors in memory
 - Load resumes only once at startup
 - Fast search across thousands of resumes
-

Phase 5 (Future AI Upgrade)

Replace TF-IDF with sentence embeddings.

```
TF-IDF
```

```
↓
```

```
Sentence Transformer
```

```
↓
```

```
Vector Search
```

```
↓
```

```
Semantic Search
```

The rest of your search pipeline (preprocessing, indexing, ranking, and returning results) remains conceptually similar, but the vector representation becomes much more powerful.

Suggested Development Plan

Since you've already completed the FAQ Search Engine, here's a practical sequence:

1. Create the project structure.
2. Add 10–20 sample resume `.txt` files.
3. Load all resumes into memory.
4. Reuse your existing NLP services (normalization, tokenization, stop-word removal, lemmatization).
5. Build a global vocabulary.
6. Compute TF-IDF vectors for all resumes.
7. Implement search by converting the user's query into a TF-IDF vector.
8. Calculate cosine similarity against every resume.
9. Sort by score and display the top matches.
10. Test with different search queries.

This project will reinforce everything you've learned in traditional NLP and prepare you for **semantic search, vector databases, and RAG**, where the ranking mechanism shifts from TF-IDF vectors to embedding vectors.